

CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 13: Templates

Course taught by: Zubair Irshad
Courtesy of some slides: Faisal Alvi

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk)



Unit Outline

1. Introduction
2. Templates
3. Summary



Motivation

We often need the **same function logic** for different data types.

```
int abs(int n) {  
    return (n < 0) ? -n : n;    // absolute value of ints  
}
```

But what if we want to handle **other types** like long or float?

```
long abs(long n) {  
    return (n < 0) ? -n : n;    // absolute value of longs  
}  
  
float abs(float n) {  
    return (n < 0) ? -n : n;    // absolute value of floats  
}
```

Problem: We're repeating the *same logic* for every type!



But why not use Function Overloading?

- **Overloading** allows multiple functions with the same name but different parameter types.
- However, you still need to **write each version manually**:
- `int abs(int n) { ... }`
- `float abs(float n) { ... }`
- `double abs(double n) { ... }`
- So, **overloading duplicates code** — it doesn't *generalize* it.



Solution: Templates

- C++ **templates** let us write one generic version that works for any type.
- Let's look at the syntax of templates!

```
template <typename T>  
T functionName(T parameter1, T parameter2, ...)   
{  
    // code  
}
```



Solution: Templates

```
template <typename T>  
T functionName(T parameter1, T parameter2, ...)   
{  
    // code  
}
```

- }T is a template argument that accepts different data types (int, float, etc.)
- typename and template are keywords.
- When an **argument of a data type** is passed to functionName(), the compiler generates a new version of functionName() for the given data type.



Calling Templates!

- Once we've declared and defined a function template, we can call it in other functions or templates (such as the main() function) with the following syntax:

```
functionName<dataType>(parameter1,  
parameter2, ... );
```



Let's look at an example!

- A template for adding two numbers!
- Note the following things:
 - How easy it is to make it work for arbitrary data types
 - How general it is!
 - How less code you have to write compared to function overloading let's say!

```
template <typename T>
T add(T num1, T num2) {
    return (num1 + num2);
}
```




How to call template functions?

- We can then call it in the main() function to add int and double numbers.

```
int main() {  
  
    int result1;  
    double result2;  
    // calling with int parameters  
    result1 = add<int>(2, 3);  
    cout << result1 << endl;  
  
    // calling with double parameters  
    result2 = add<double>(2.2, 3.3);  
    cout << result2 << endl;  
  
    return 0;  
}
```



Another way to call template functions!

- This is implicit deduction!

```
int main() {  
    int result1;  
    double result2;  
  
    // Compiler automatically deduces T = int  
    result1 = add(2, 3);  
    cout << result1 << endl;  
  
    // Compiler automatically deduces T = double  
    result2 = add(2.2, 3.3);  
    cout << result2 << endl;  
  
    return 0;  
}
```

What happens under the hood!

```
#include<iostream>
```

```
template<typename T>
```

```
T add(T num1, T num2) {  
    return (num1 + num2);  
}
```

```
int main() {  
    ... ..
```

```
    result1 = add<int>(2,3);  
    ... ..
```

```
    result2 = add<double>(2.2,3.3);  
    ... ..
```

```
}
```

```
int add(int num1, int num2) {  
    return (num1 + num2);  
}
```

```
double add(double num1, double num2) {  
    return (num1 + num2);  
}
```



The Big Picture!

- The key innovation in function templates is to represent the data type by a name that can stand for any type.
- In the examples we saw for function template, this name is T.
- The template keyword signals the compiler that we're about to define a function template.
- Throughout the definition of the template we substitute the template argument, T.
- **Key Takaway:**
 - Function templates provide one function that can work for multiple data types
 - Compiler automatically generates type-specific code



Template Functions – Multiple Arguments!

- `template <typename T1, typename T2>` → **declares two independent** template parameters.
- The compiler **automatically** deduces their types from the arguments.
- Works even when argument types differ (e.g., `int + double`).
- The expression `(a + b)` performs implicit type conversion as per C++ rules (e.g., `int → double`).
- **What is the output?**

```
#include <iostream>
using namespace std;

// Template with two different type parameters
template <typename T1, typename T2>
void showSum(T1 a, T2 b) {
    cout << "Sum = " << (a + b) << endl;
}

int main() {
    // Compiler deduces T1 = int, T2 = double
    showSum(5, 2.5);

    // Compiler deduces T1 = double, T2 = float
    showSum(3.1, 4.2f);

    // Compiler deduces T1 = int, T2 = int
    showSum(2, 8);

    // Compiler deduces T1 = double, T2 = int
    showSum(8.5, 2);

    return 0;
}
```

Template Functions – Multiple Arguments!

- **What is the output?**

- Sum = 7.5
- Sum = 7.3
- Sum = 10
- Sum = 10.5

```
#include <iostream>
using namespace std;

// Template with two different type parameters
template <typename T1, typename T2>
void showSum(T1 a, T2 b) {
    cout << "Sum = " << (a + b) << endl;
}

int main() {
    // Compiler deduces T1 = int, T2 = double
    showSum(5, 2.5);

    // Compiler deduces T1 = double, T2 = float
    showSum(3.1, 4.2f);

    // Compiler deduces T1 = int, T2 = int
    showSum(2, 8);

    // Compiler deduces T1 = double, T2 = int
    showSum(8.5, 2);

    return 0;
}
```

Template Functions – Multiple Arguments!

- Note keywords `typename` and `class` are used interchangeably. **`typename`** is newer notation!
 - **`template <class T>`** and **`template <typename T>`** are equivalent
- **`ArrayType`** → type of the array elements (e.g., `int`, `double`). For instance `T1` in the previous example
- **`ValueType`** → type of the value being searched for (can be different). For e.g. `T2` in the previous example.
- The compiler **deduces both types automatically** from the arguments.
- The function works for `int`, `double`, `char`, or even custom classes (if `==` is defined).

```
template <class ArrayType, class ValueType>
int find(ArrayType* array, ValueType value, int size)
{
    for (int j = 0; j < size; j++)
        if (array[j] == value)
            return j;
    return -1;
}
```



Template Functions – Multiple Arguments!

```
int main()
{
    int intArray[] = {10, 20, 30, 40, 50};
    double dblArray[] = {1.1, 2.2, 3.3, 4.4};

    cout << "Index of 30: " << find(intArray, 30, 5) << endl;           // int array, int value
    cout << "Index of 2.2: " << find(dblArray, 2.2, 4) << endl;         // double array, double value
    cout << "Index of 30 in dblArray: " << find(dblArray, 30, 4) << endl; // double array, int value

    return 0;
}
```

- What is the output?

Template Functions – Multiple Arguments!

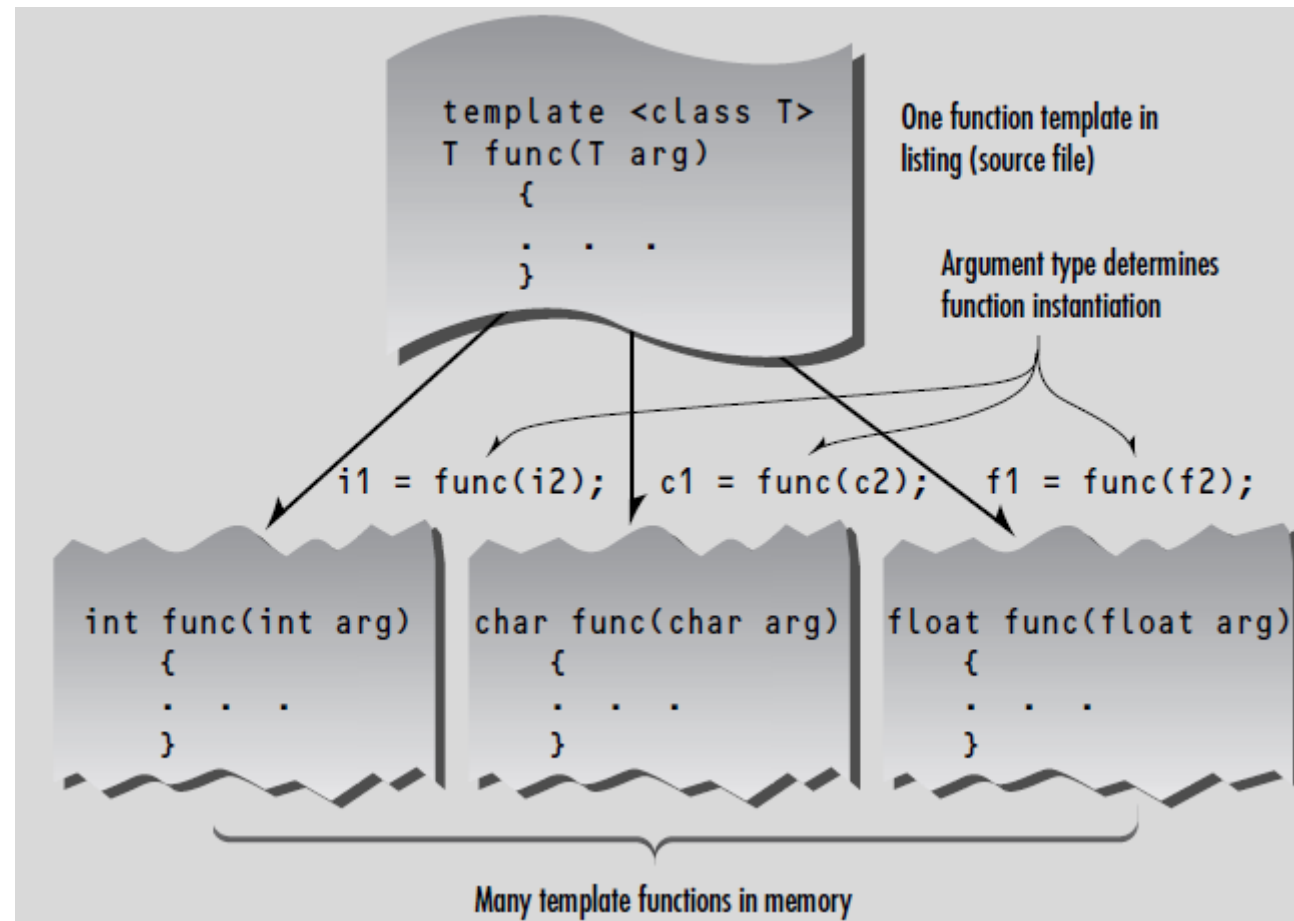
```
int main()
{
    int intArray[] = {10, 20, 30, 40, 50};
    double dblArray[] = {1.1, 2.2, 3.3, 4.4};

    cout << "Index of 30: " << find(intArray, 30, 5) << endl;           // int array, int value
    cout << "Index of 2.2: " << find(dblArray, 2.2, 4) << endl;         // double array, double value
    cout << "Index of 30 in dblArray: " << find(dblArray, 30, 4) << endl; // double array, int value

    return 0;
}
```

- What is the output?
 - Index of 30: 2
 - Index of 2.2: 1
 - Index of 30 in dblArray: -1

Templates: Multiple Arguments





Function Templates vs Class Templates

Function Templates	Class Templates
Make one function work for many data types	Make one class work for many data types
Example: template <typename T> T add(T a, T b)	Example: template <typename T> class Stack { ... };
Compiler generates type-specific versions when called	Compiler generates type-specific classes when used

In Short:

- Function templates **generalize behaviors**
- Class templates **generalize data structures**



An example of class Templates!

- Let's implement a Stack Class as a template! (Luckily the declaration of template stays the same for us)

```
template <typename Type>
// Note that Type here can be arbitrary
i.e. int, double, string,
class Stack {
private:
    Type st[MAX]; // stack: array of any type
    int top;      // index of top element

public:
    // Constructor
    Stack() : top(-1) {} // initializer list
```

An example of class Templates!

- Let's implement a Stack Class as a template! (Luckily the declaration of template stays the same for us)

```
// Push an item onto the stack
```

```
void push(Type var) {  
    if (top == MAX - 1) {  
        cout << "Stack overflow!" << endl;  
        return;  
    }  
    st[++top] = var; // increment first, then use the new value.  
}
```



An example of class Templates!

- Let's implement a Stack Class as a template! (Luckily the declaration of template stays the same for us)

```
// Pop an item from the stack
```

```
Type pop() {  
    if (top == -1) {  
        cout << "Stack underflow!" << endl;  
        return Type(); // return default value for the type  
    }  
    return st[top--]; //use the current value, then decrement after.  
}
```



An example of class Templates!

```
int main() {  
    // Create stacks of different types using the same template  
    Stack<int> intStack; // stack that stores integers  
    Stack<string> stringStack; // stack that stores strings  
  
    // Using Stack<int>  
    intStack.push(10);  
    intStack.push(20);  
    cout << "Popped from intStack: " << intStack.pop() << endl;  
  
    // Using Stack<string>  
    stringStack.push("Hello");  
    stringStack.push("World");  
    cout << "Popped from stringStack: " << stringStack.pop() << endl;  
    return 0;}
```



An example of class Templates!

```
int main() {  
    // Create stacks of different types using the same template  
    Stack<int> intStack; // stack that stores integers  
    Stack<string> stringStack; // stack that stores strings  
  
    // Using Stack<int>  
    intStack.push(10);  
    intStack.push(20);  
    cout << "Popped from intStack: " << intStack.pop() << endl;  
  
    // Using Stack<string>  
    stringStack.push("Hello");  
    stringStack.push("World");  
    cout << "Popped from stringStack: " << stringStack.pop() << endl;  
    return 0;}
```


Analogy: Function Template vs Class Template

Concept	Function Template	Class Template
Definition	<pre>template <typename T> T add(T a, T b)</pre>	<pre>template <typename T> class Stack { ... };</pre>
Purpose	One logic for many data types	One class for many data types
Compiler action	Generates functions like <code>add(int, int)</code>	Generates classes like <code>Stack<int></code>
Usage	<code>add(2, 3)</code>	<pre>Stack<int> s; s.push(5);</pre>



Class Exercise!

- Create a **class template** Dictionary that stores key-value pairs of *any types* (like a mini version of a map).
- The class should:
 - Use two template parameters — one for the key type and one for the value type.
 - Store keys and values in two parallel arrays.
 - Allow you to:
 - Add a key-value pair.
 - Search for a value by key.
 - Display all pairs.



Conclusion

- Templates provide generic versions of functions and classes.
- The compiler creates concrete versions of your class or function based on the type you use.
- We can think of templates as generic code generators at runtime!